

ASSIGNMENT 1

1.1

Intelligent systems are advanced machines designed to perform tasks useful to humans, often automating those that require physical or mental effort. They represent a type of system derived from successful applications of artificial intelligence (AI). The capabilities of these tools have progressively increased, allowing them to perform tasks requiring increasing levels of intelligence.

1. Characterization of Intelligent Systems: External Behavior and Core Properties

An intelligent system can be characterized based on its observable external behavior, which often aligns with the common approach in AI describing an intelligent machine as a system that operates as an agent and behaves rationally.

- **Operating as an Agent:** This means the system is capable of **perceiving its environment, acting within that environment, and interacting with other agents**, such as human users or other artificial systems. This interaction involves sensing the environment (e.g., through real or virtual sensors) to gather data about its state and acting in it (e.g., via actuators). The system is considered "situated" because it operates in a continuous, close-coupled interaction of sensing and acting.
 - **Interaction with the Environment:** Intelligent systems utilize sensors to obtain measurements about the environment's physical magnitudes (like temperature, images, or pressures) or their own body's state (e.g., battery charge). They generate a "world model" by integrating features extracted from sensors, which dynamically updates to reflect the environment's state. This process must handle difficulties like incomplete or noisy sensor information and the need to fuse data from multiple sensor types.
 - **Interaction with Other Agents:** Intelligent systems often interact with human users, fulfilling roles such as a "**delegate**" or an "**advisor**".
 - In a **delegate role**, the user entrusts a task to the system, and the system autonomously performs it in the environment, making its own decisions without user intervention. An example is a self-driving car choosing a path and controlling driving mechanisms to transport a passenger.
 - In an **advisor role**, the user retains decision-making authority over actions in the environment, and the system provides useful information or advice

to facilitate these decisions. This can include planning, design, resource allocation, detecting problems, diagnosing causes, or predicting future states.

- Systems can also be **proactive**, initiating tasks based on their own goals or environmental perceptions, rather than waiting for user requests.
- **Rational Behavior:** An intelligent system exhibits rational behavior in two ways:
 - **Acting Rationally:** The system performs actions that **maximize its chances of success** or optimize a predefined performance measure. An external observer can conclude a machine acts rationally if it consistently takes actions that successfully optimize this measure. This aligns with the idea that intelligence is an ability to achieve goals.
 - **Thinking Rationally:** The system is able to **provide justified reasons that explain its beliefs or actions**. This involves using understandable language to describe its knowledge and how conclusions are reached, often through logical, approximate, or analogical reasoning. This capability is crucial for "explainable artificial intelligence" (XAI), fostering user trust and helping identify wrong behaviors.
- **Learning:** Learning is a key characteristic of intelligent behavior, understood in two forms:
 - **Performance-based learning:** A system learns if it improves its performance at tasks with experience, as measured by a performance metric. This is the basis of machine learning.
 - **Knowledge acquisition learning:** An agent learns by acquiring new beliefs, which can be verified if the agent can describe its knowledge to an observer. This can happen by observing the environment, reasoning from existing beliefs, or through knowledge transfer from other agents using language. Learning enables adaptation to environmental changes, both individually and socially.

The technology behind true AI is still in its infancy, with many current systems being "pseudo-A.I." that rely on advanced machine learning with extensive behavioral algorithms, adapting to user preferences based on large datasets rather than truly "getting smarter in the existential sense". However, the evolution and rise of AI are unstoppable. The inevitable rise of **quantum computing** is expected to make AI systems smarter, faster, and more human-like, potentially solving complex global problems but also posing inherent risks to digital security if not managed with quantum-resistant cryptography.

2. Functional Components of Intelligent Systems

Beyond external behavior, intelligent systems can be broken down into main functions that generate their intelligent behavior:

- **Environment Understanding:** This function involves interpreting data from the environment to form a representation of the current situation and evaluating it within a broader context.
 - **Environment Data Acquisition:** Obtaining necessary measurements from sensors (e.g., images, temperatures) or receiving information from other agents. This includes filtering data, active data gathering, sensor control, and monitoring for errors or malfunctions.
 - **World Modeling:** Creating and dynamically adapting representations of the world from sensor data, often using hierarchical organizations and task-specific representations.
 - **Situation Analysis:** Inferring unobserved facts, diagnosing causes of current situations, or predicting future states based on background knowledge.
 - **Assigning Value to the Situation:** Assessing how well a situation fits user desires or system objectives, often by computing deviations from desired measurements or using complex evaluation functions.
- **Acting in the Environment:** This includes deciding what actions to perform and executing those actions.
 - **Reactive Decision:** Deciding actions as an immediate response to current observations, suitable for rapid responses to environmental changes, like a self-driving car stopping for an obstacle.
 - **Deliberate Decision:** Making decisions through a reasoning process based on goals, distinguishing between **goal reasoning** (determining what goals to achieve) and **reasoning about actions** (how to achieve those goals). This involves analyzing alternative actions, often by optimizing a performance measure. Examples include planning, configuration, assignment, and scheduling.
 - **Execution Control:** Systems like autonomous robots can execute actions, refining high-level plans into detailed commands for actuators, adapting to dynamic situations. This often involves control theory and self-configuration capabilities.
 - **Execution Monitoring:** Supervising the correct execution of actions by checking preconditions, verifying progress, detecting completion, and identifying failures.

- **Communicating with Others:** Intelligent systems can send and receive messages via various means like speech, text, or graphical representations. This facilitates distributed task performance in complex social organizations, allowing agents to share information and update beliefs about the environment and other agents. Communication involves functions like speech recognition, natural language interpretation, dialog management, and natural language generation.

3. Building Intelligent Systems and Associated Challenges

Developing intelligent systems typically combines several AI methods, categorized into **connectionist approaches** (based on neural computation, like neural networks) and **symbolic approaches** (based on formal logic, probability, or hybrid methods). Symbolic methods are often used for conscious cognitive abilities and represent beliefs with human-understandable symbols, while connectionist methods are used for sensorimotor skills and language models, often learned automatically from data.

Automatic methods like machine learning and natural language processing (NLP) are used to build components of intelligent systems offline (as developer tools) or online (as continuous learning/communication abilities within the system).

However, evaluating intelligent systems presents issues:

- **Model Understandability:** Models generated by connectionist machine learning methods (neural networks) are not directly understandable by developers, making it hard to trace reasoning or identify causes of wrong decisions. Neurosymbolic approaches or statistical methods within explainable AI are proposed to address this.
- **Ethical Concerns:** Systems must operate in accordance with ethical values like fairness and avoiding harmful use. Biased training data can lead to unfair operations, as seen with Microsoft's Tay chatbot.
- **Knowledge Acquisition Bottleneck:** First-generation expert systems, like Mycin for medical diagnosis, relied on manual knowledge acquisition, which was labor-intensive and limited the size of knowledge bases. Modern systems aim for more automated data acquisition from unstructured texts or sensors.
- **Unstructured Data:** A significant challenge is converting vast amounts of existing unstructured data, information, and knowledge into structured forms that intelligent systems can use.

- **Job Displacement:** AI's increasing incorporation into industries raises concerns about potential job displacement due to automation. While AI excels at automating tasks, humans still outperform AI in tasks requiring creativity, empathy, critical reasoning, or understanding nuanced language.

4. Powerful Examples of Artificial Intelligence in Use Today

AI applications have significantly evolved, revolutionizing industries and finding uses in nearly every business sector.

- **E-commerce:** AI enhances shopping experiences and optimizes operations through **recommendation engines** (e.g., **Amazon** uses AI to suggest products based on browsing history and preferences) and **AI-driven chatbots** for customer support.
- **Lifestyle:** AI is integrated into various applications, from personal assistants to smart home devices, simplifying daily tasks and managing schedules. **Siri** (Apple's voice-activated personal assistant) and **Alexa** (Amazon's smart home hub) use machine learning to understand natural language questions, find information, give directions, set reminders, and control smart home devices. **Nest** thermostats learn temperature preferences for optimal comfort and energy savings.
- **Navigation and Autonomous Systems:** AI improves navigation systems with real-time traffic data, weather conditions, and historical patterns for route suggestions (e.g., **Google Maps**). It powers **autonomous vehicles** (e.g., **Tesla's Autopilot**) for self-driving features, predictive capabilities, lane-keeping, adaptive cruise control, and parking, receiving over-the-air updates to get smarter. Robotics, including **industrial collaborative robots** and **domestic robots**, also leverage AI for complex or dangerous tasks.
- **Natural Language Processing (NLP):** Enables machines to understand, interpret, and generate human language, facilitating **translation, sentiment analysis, and voice-activated assistants**. **Grammarly** uses NLP for writing improvement, and **ChatGPT** generates human-like text responses for content creation and chatbots.
- **Computer Vision and Face Recognition:** AI allows machines to interpret visual information, recognizing objects, people, and activities in images and videos. This is used in self-driving cars for obstacle detection and in security systems for **face recognition** (e.g., **Apple's Face ID**).
- **Healthcare:** AI improves diagnostics, personalizes treatment plans, and optimizes patient care by analyzing medical images, predicting disease outbreaks, and assisting in drug discovery (e.g., **IBM Watson Health**).

- **Gaming:** AI creates realistic and engaging experiences by generating intelligent behavior in non-player characters (NPCs) and adapting to player actions.
- **Social Media:** AI personalizes content feeds, detects fake news, and improves user engagement (e.g., **Facebook** curating news feeds, **Snapchat** enhancing AR filters).
- **Marketing:** AI helps businesses understand customer behavior, optimize campaigns, and deliver personalized experiences by analyzing data to identify trends and automate content delivery (e.g., **Mailchimp** for email marketing).
- **Chatbots:** AI-powered chatbots provide instant customer support, answering queries and assisting with tasks around the clock.
- **Finance:** AI improves risk management, fraud detection, and investment strategies (e.g., **robo-advisors like Betterment**).
- **Entertainment:** AI personalizes content recommendations (e.g., **Netflix** analyzes viewing habits, **Spotify** recommends music), creates realistic visual effects, and enhances audience engagement. **Pandora** uses AI's "musical DNA" based on 400 musical characteristics to recommend songs.
- **Customer Support:** Companies like **Cogito** use machine learning and behavioral science to improve the emotional intelligence of customer support representatives, enhancing voice call interactions.
- **Travel Industry:** **Boxever** leverages machine learning to improve customer experience in the travel industry, while **John Paul**, a luxury travel concierge, uses potent AI in predictive algorithms to understand client desires.
- **Management Support Systems:** AI assists operators in managing complex dynamic environments, like urban bus transportation networks, by collecting sensor data, modeling the environment, analyzing situations, evaluating severity, and recommending actions.

The global AI market is experiencing significant growth, projected to reach \$1.8 trillion by 2030, and is poised to contribute substantially to the world economy. This growth underscores the increasing demand for skilled professionals in this dynamic field.

1.2.

1. Intelligent systems are a rapidly evolving technology that has become increasingly vital in recent years, addressing the demands of accelerated technological growth and the

interconnectedness of our world. These systems are designed to **solve complex problems automatically and more efficiently within specific environments.**

At their core, intelligent systems represent a collaboration between people and various advanced technologies. Key technologies that contribute to their formation include **Big Data, the Internet of Things (IoT), mobile networks (3G, 4G, 5G), Artificial Intelligence (AI), robotics, video analytics, computer vision, and augmented reality.** The inherent "intelligence" of these systems stems from the constant interconnectivity, interaction, and learning among their diverse physical, digital, and human components, marking a significant aspect of Industry 4.0's transformation.

Generally, intelligent systems function by utilizing **IP (Internet Protocol) technology and sensors to collect information** from their environment. This collected data is then shared among their different elements to achieve a common objective. This crucial interconnection between the digital and physical realms is known as the Internet of Things (IoT). **Big Data** further enables these systems by allowing the collection of vast amounts of information and knowledge. Crucially, intelligent systems learn from these experiences through artificial intelligence technologies, primarily machine learning.

2. Most Impressive Definition

Among the various definitions provided, the one offered by Martin Molina in his paper, "What is an intelligent system?" published by the Technical University of Madrid, is the most impressive due to its comprehensive nature and depth:

"An intelligent system operates in an environment with other agents, possesses cognitive capabilities such as perception, action control, deliberative reasoning or language, follows principles of behavior based on rationality and social norms, and has the ability to adapt by learning."

This definition stands out because it not only highlights the system's operational context ("in an environment with other agents") but also enumerates the **specific cognitive capabilities** it must possess, such as "perception, action control, deliberative reasoning or language". Furthermore, it emphasizes the importance of the system adhering to "principles of behavior based on rationality and social norms" and its crucial ability to "adapt by learning". This detailed explanation moves beyond merely stating that a system incorporates intelligence or automates tasks, providing a more holistic understanding of the complex functionalities and behavioral frameworks that truly define an intelligent system. It covers both the technical capabilities and the broader implications of its interaction within an environment.

3. Examples of Intelligent Systems

Intelligent systems are integrated into nearly every aspect of daily life, often working behind the scenes. Here are numerous examples:

- **Maps and Navigation:** Applications like Google Maps or Apple Maps use AI, specifically machine learning algorithms, to provide optimal routes, identify road barriers, and understand traffic congestion. They remember building edges for better visuals and recognize house/building numbers, as well as identify traffic flow changes to recommend routes avoiding roadblocks.
- **Facial Detection and Recognition:** These systems are used for features like virtual filters on photos and Face ID for unlocking phones, as well as for surveillance and security in government facilities and airports. Face detection identifies any human face, while face recognition identifies a specific face.
- **Text Editors or Autocorrect:** Tools like Grammarly leverage AI algorithms, including machine learning, deep learning, and natural language processing (NLP), to identify incorrect language usage and suggest corrections in various written mediums. They are taught grammar through high-quality language data.
- **Search and Recommendation Algorithms:** Online platforms like movie streaming services or e-commerce sites use smart recommendation systems that learn user behavior and interests over time by tracking online activity. This data is collected, stored, and analyzed using machine learning and deep learning to predict user preferences and offer relevant suggestions.
- **Chatbots:** These AI solutions are used by companies to improve customer service efficiency. Programmed algorithms enable chatbots to answer frequently asked questions, take and track orders, and direct calls. They are trained through NLP to impersonate customer representative conversational styles and can answer complex questions, even self-correcting based on user feedback.
- **Digital Assistants:** Virtual assistants such as Siri utilize NLP, ML, statistical analysis, and algorithmic execution to understand voice commands and perform tasks, like calling a contact. Voice and image search also operate similarly.
- **Social Media:** AI algorithms are deployed to monitor content, suggest connections, and serve targeted advertisements to users, ensuring engagement. They can swiftly detect and remove problematic posts through keyword identification and visual image

recognition, often utilizing the neural network architecture of deep learning. Social media AI also understands user content preferences and suggests similar content.

- **E-Payments:** Banks leverage AI to simplify payment processes, enabling deposits, money transfers, and account opening from anywhere. AI provides security, identity management, and privacy controls, and can detect potential fraud by observing unusual credit card spending patterns.
- **Weather Forecasting:** AI tools, such as Google's DeepMind GraphCast model and Nowcast, make real-time predictions based on historical data, parsing decades of observed weather data much faster than humans.
- **Music Mastering:** Online music companies are incorporating AI to automate the final step of music production, controlling volume and frequency characteristics to ensure acceptable sound quality across various conditions and devices.
- **Autonomous Robots:** These complex intelligent systems, initially for industrial operations, are increasingly found in home and automotive markets, promising new areas of collaboration for high-risk situations.
- **Computer Vision:** This application gives computers the ability to understand visual information from images or videos using neural network-based algorithms. It includes:
 - **Object Classification:** Identifying the type of object in an image without necessarily locating it.
 - **Object Detection:** Locating and classifying objects, used in autonomous cars to detect and classify traffic signs.
 - **Object Tracking:** Knowing the location and type of each object, useful for searching vehicles or people.
 - **Human Body Understanding:** Researchers are teaching computers to "see" human bodies by analyzing surfaces, movements, and deformations using 360° body scanners, allowing them to track complex movements and even infer hidden body parts. Applications include assisting individuals with anorexia nervosa, creating clothing programs for the fashion industry with realistic fabric behavior on avatars, and generating precise digital 3D avatars for the film industry.
- **Expert Systems:** These systems use accumulated knowledge and past experience to make deductions and simulate problem-solving strategies, mirroring the human brain's

functioning through neural networks. They are used for challenging problems like analysis, treatment, and diagnostics in various fields.

- **Sentimental Analysis:** A field within AI and applied linguistics that analyzes interactions between machines and humans to recognize concepts and intentions in written messages, often used on social networks. It classifies texts (e.g., product reviews) as positive, negative, or neutral to help companies understand customer reactions.
- **Network Monitoring and Analysis Systems:** Software that monitors and automates network bandwidth and components (firewalls, routers, devices, servers) to ensure proper functioning. They optimize information flow, identify performance issues, and organize data in a user-friendly interface.
- **Smart Watches (Biometric Monitoring):** These popular devices connect to other devices (like smartphones) or directly to networks, enabling calls, emails, music, file management, and web access. Crucially, they can also monitor biometrics such as heart rate, blood pressure, oxygenation, and physical activity, which is vital for health management.
- **AI Visual Inspection:** Implemented in factories, especially in assembly lines, to detect defects and problems in products. This system uses computer vision and deep learning to simulate human visual examination, learning patterns to identify defects, thereby minimizing human error and saving time.
- **Intelligent Manufacturing Systems (IMS):** These systems involve collaboration between humans, machines, and processes to optimize product planning and delivery within manufacturing. AI within IMS can create models for planning, factory layout, and production design, and even develop new products and plans without human intervention, while also identifying errors in existing processes.
- **Crime Detection (Intelligent Surveillance Systems):** Used to enhance public and private safety by connecting humans, computers, software, cameras, and the internet to create a surveillance network that detects potential crimes in real-time. AI monitoring eliminates human fatigue, improves chances of catching criminals, learns to recognize patterns and social behaviors, and utilizes face-recognition to flag illicit activities for authorities. For example, vehicleDRX can identify a vehicle from only about 5% of a camera image, significantly boosting productivity at crime centers.
- **Functional Restoration (Assistive Technologies):** Intelligent systems are developed to create assistive technologies that help people perform tasks they can no longer do or never could, often due to injury, degenerative disease, or old age. Examples include:

- **Robotic exoskeletons** to aid paralyzed individuals in walking.
- **Cochlear implants** to help deaf individuals hear.
- **Robotic arms** designed to assist individuals with muscular diseases in performing daily living activities, such as brewing coffee. These systems incorporate augmented reality for the human-machine interface, machine perception to recognize objects, inference of user intent, and calculations for robotic arm movements. The aim is to increase both independence and the ability to help others.

In conclusion, intelligent systems are pervasive, integrated technologies that leverage AI and other advanced tools to perform complex tasks, learn from experience, and enhance various aspects of human life and industry.

1.3. Core Areas of Intelligent System Applications and AI Techniques

1. Customer Service and Business Operations AI is significantly transforming customer service and internal business operations by automating tasks and enhancing interaction quality.

- **Chatbots and Virtual Assistants:** AI-powered chatbots like **Kustomer's** offering can directly engage customers, providing personalized answers and resolving issues automatically. Similarly, **Navan's** Ava AI chatbot handles customer service issues, including user questions, trip bookings, flight changes, and spend data analysis. **IBM's** watsonx can serve as an employee Q&A resource or a customer service chatbot. **EliseAI** provides an AI-powered assistant for marketing teams, managing prospect and customer communication via email, texts, and calls, including rescheduling meetings and sending follow-ups. **Cash App** uses complex **natural language processing (NLP)** techniques to develop its customer support chatbots. These assistants free employees from unnecessary labor and provide quick information retrieval.
- **Productivity and Collaboration Tools:** **Microsoft's Cortana** serves as a "personal productivity assistant" integrated into Microsoft 365, reminding users about emails or recognizing voice instructions to join meetings. **Atlassian Intelligence** boosts workplace productivity across its platform, including Jira, Trello, and Rovo. **Notion** offers AI tools for text generation, arithmetic, results predictions, summarizing information, fixing grammar, and analyzing large text datasets for insights. **Dropbox Dash** is an AI-powered search tool that summarizes and organizes content from various sources into a single dashboard. **Grammarly** provides AI-powered, contextually relevant writing assistance, helping teams increase productivity and save costs.

- **Enterprise Resource Planning (ERP) and Workflow Automation: SquareWorks Consulting** integrates AI through its Invoice AI tool for AP automation scaling in ERP solutions. **Altana's** "Value Chain Management System" uses AI to improve workflows within its public-private network. **Cisco Meraki's** cloud-based platform leverages AI capabilities to streamline network management for remote and hybrid work environments.
- **Risk Management and Compliance: LogicGate's** Risk Cloud solution helps businesses operationalize and automate risk compliance, providing an interconnected view of risk.
- **Quality Assurance: MaestroQA** utilizes AI to analyze data for incidents and trends in quality assurance software, operating at a scale and speed unreplicable by human analysts.
- **Consulting: Quantum Rise** is an AI consulting company focused on delivering measurable results like reduced costs and increased revenue by creating continuously improving products. **PwC** consults on tech solutions, having integrated AI into an auto insurance provider's claims estimates process to improve operational efficiency and customer experience.

2. Robotics and Automation AI-powered robots are capable of solving problems and "thinking" in a limited capacity, undertaking increasingly complex tasks.

- **Automated Cleaning and Home Devices: iRobot's Roomba** vacuum uses AI to scan room size, identify obstacles, remember efficient cleaning routes, and determine vacuuming needs without human assistance. **Softbank Robotics** developed Pepper, a humanoid robot with an "emotion engine" that recognizes faces and basic human emotions, and NAO, a bipedal robot for education and research, as well as Whiz, an autonomous vacuum for commercial cleaning.
- **Manufacturing and Industrial Automation: Manufacturing robots** automate processes in factories. **GrayMatter Robotics** uses AI to power robotic systems for manufacturing tasks like sanding, grinding, spraying, buffing, and quality inspection, suitable for high-mix manufacturing environments. **Machina Labs** creates smart factories using robots for building and assembling components in an automated environment, specializing in formed sheet metal goods for various sectors. **AMP** designs robotic systems for recycling sites, where AI-powered robots quickly discern material variations (type, shape, texture, color, logos) to efficiently process objects on conveyor belts.

- **Autonomous Vehicles and Transportation:** Self-driving cars are equipped with sensors that capture vast amounts of data—car speed, road conditions, pedestrian whereabouts, other traffic—and use AI to interpret this data and act accordingly. **Tesla** develops and enhances its electric vehicles' autonomous driving capabilities (automatic braking, lane changes, parking) using AI and robotics programs, including neural networks and autonomy algorithms. **Waymo**, Google's self-driving car project, uses advanced sensors and AI to decipher data, analyze situations, and make safe predictions for optimal next moves. **Intelligent transportation** is a key application area.
- **Automated Checkout and Payments:** **Whole Foods Market** utilizes Amazon's Just Walk Out system, which employs **computer vision, sensor fusion, and deep learning** to track items, build a virtual shopping cart, and allow customers to leave without traditional checkout. **Metropolis Technologies** offers a computer vision platform for automated parking payments, where drivers simply drive in and out after connecting their vehicle. **Instacart's Caper Cart technology** facilitates an intelligent brick-and-mortar retail experience by recognizing items, weighing produce, and accepting payment.

3. Healthcare AI is a game-changer in healthcare, improving efficiency and enabling innovation.

- **Healthcare Management:** AI-enabled virtual assistants can reduce unnecessary hospital visits and free up nurses' time.
- **Drug Discovery and Development:** Pharmaceutical companies like **Takeda** are incorporating AI to make drug discovery more efficient.
- **Personalized Health:** **Well** develops personalized health plans using AI, collecting data on conditions and knowledge gaps to provide coaching, tips, and guidance for screenings, prescriptions, and doctor visits.
- **Patient Communication:** **Artera** leverages AI on its platform to help healthcare professionals make data-driven decisions and offers AI agents that use **large language, text-to-speech, speech-to-text, and generative voice models** for human-like patient interaction.
- **Medical Technology and Safety:** **Greenlight Guru** provides cloud-based solutions for medical technology, using an AI search engine to aggregate and process industry data, detect security risks, and automatically calculate risk assessments. **Motorola Solutions** builds AI-enabled assistive technologies for public safety, including conversational AI and **natural language processing (NLP)** to search databases, provide information from voice commands, and transcribe 911 calls in real time.

- **Research and Development:** **Liberty Mutual Insurance's** partnership with MIT focuses on AI research to modernize the insurance industry, exploring how **computer language understanding** could speed up claims processing and increase transparency for decision-making algorithms.
- **General Medical Care:** AI is broadly applicable in **medical care**.

4. Finance The financial sector heavily relies on accuracy and high-volume data processing, making AI a natural fit for applications like automation, chatbots, and anti-fraud defenses.

- **Automated Financial Investing:** AI supports automated financial investing. **Northwestern Mutual** is significantly investing in AI to improve customer experience and data management efficiency in financial services. **Morningstar** uses AI to power its Intelligence Engine, simplifying the process of tracking investment data, and is beta testing "Mo," a chatbot serving as a digital research assistant.
- **Fraud Detection and Risk Mitigation:** AI algorithms detect financial fraud. **Riskified** uses AI to analyze transaction and consumer data to assess the likelihood and identify instances of policy abuse, fraud, and chargebacks, with AI analysts monitoring traffic for anomalies and organized fraud. **Gradient AI** offers claim and underwriting automation solutions for the insurance industry, relying on **AI and machine learning models** to manage routine tasks and provide predictions that save time and money. **Cash App** uses **machine learning and AI** to mitigate risk and prevent fraud on its platform.
- **Alternative Investments:** **Canoe** automates alternative investments, allowing users to create workflows for analysis and data collection, access data, and handle document and data extraction.
- **Financial Services AI Platforms:** **Hebbia AI** provides an AI platform for financial services, allowing users to build and deploy custom AI agents that extract, structure, and analyze large numbers of documents for efficient workflows.

5. Travel and Transportation AI is significantly contributing to the global travel technologies market by facilitating human-like interactions, better booking prices, and travel recommendations.

- **Travel Booking and Management:** AI can act as a virtual travel booking agent. **Navan's** Ava AI chatbot assists with trip bookings and flight changes.

- **Route Planning and Navigation:** **Google Maps** uses AI to scan road information and algorithms to determine optimal routes for various modes of transport, integrating voice assistants and augmented reality maps for real-time guidance.
- **Fleet Management:** **Motive** uses AI to support its fleet management software for logistics companies, enabling live GPS tracking, smart dash cams, and detecting dangerous driving in real-time, leading to substantial reductions in driver accidents. **Intelligent transportation** systems are a key application.

6. Social Media and Communication AI is highly valuable in social media for organizing data, recognizing images, introducing chatbots, and predicting cultural shifts.

- **Content and User Management:** **Hinge** uses AI to power its recommendation algorithms for profile matching based on metrics, demographics, and engagement. **Meta** develops open-source AI models like Llama 4, integrated into its social platforms, with features like multimodal understanding, advanced reasoning capabilities, and real-time image recognition.
- **Chatbots:** **Snap Inc.'s** My AI chatbot helps users with trivia questions, trip suggestions, or gift ideas.

7. Retail and E-commerce AI-driven algorithms personalize user experiences, increase sales, and build customer loyalty in retail and e-commerce.

- **Product Recommendations:** **Amazon** extensively uses AI for product recommendations.
- **Checkout-free Shopping:** As mentioned, **Whole Foods Market** uses Amazon's Just Walk Out system for a checkout-free experience. **Instacart's Caper Cart** also facilitates this.
- **Warehouse and Supply Chain Management:** **Amazon** employs warehouse robots that use AI to grab, sort, and ship products. AI also optimizes supply chains for businesses.
- **Personalized Shopping and Ordering:** **Hungryroot** uses AI to create personalized meal recommendations based on user input and activity.
- **Optimizing Operations:** **McDonald's Global Technology** uses AI for automated drive-thru operations and continuously optimizing digital menu displays based on factors like time of day, restaurant traffic, and item popularity.

8. Marketing and Advertising Marketers are increasingly allocating budgets to AI for campaign management, leveraging tools like keyword search technologies, chatbots, and automated ad buying.

- **Targeted Messaging and Lead Conversion:** **Ylopo** provides an AI-powered digital marketing platform for real estate professionals, using Ylopo AI Text and AI Voice products to target and convert leads. **Klaviyo** uses embedded AI to comb through consumer data, offering predictive insights on metrics like order frequency, lifetime value, and churn rate, then generating and implementing personalized messaging campaigns. **Zeta Global** incorporates predictive AI into its cloud-based platform to help businesses target the right customers and recommend actions for customer acquisition and retention.
- **Campaign Optimization and Ad Placement:** **GumGum** uses AI to optimize and automate advertising. Its contextual intelligence platform Verity analyzes video, audio, image, and written content to ensure relevance and determines ideal ad placements based on context, rather than third-party cookies. **RTB House** goes beyond basic AI-powered campaigns by informing them with **deep learning algorithms** to compile complex data, segment audiences, and determine best platforms. **Smartly** uses AI to streamline the creation and execution of optimized media campaigns, offering automated ad changes triggered by performance. **System1's** omnichannel digital marketing platform uses proprietary AI and **machine learning algorithms** for customer acquisition. **Optimum's** AI-enabled media planning tool, Alice, streamlines the process of plotting media campaign strategies to reach target audiences.

9. Education and Learning AI provides personalized learning options for students and facilitates employee training.

- **Learning Assistance:** **Duolingo's** conversational chatbot Lily (powered by GPT-4) provides learning assistance, generated feedback for mistakes, and enables roleplay scenarios for real-world conversations.
- **Employee Training and Upskilling:** **Skillsoft's** Conversation AI Simulator (CAISY) lets users practice business and leadership conversations, simulating the other side and giving feedback. **360Learning's** AI-powered platform allows employers to develop custom course content and enable personalized employee upskilling pathways.
- **Test Preparation:** **Blueprint Test Preparation's** AI platform helps students engage with practice questions and exams, determining why they get questions wrong and helping them learn from mistakes.

- **General Education and Entertainment:** Intelligent systems are applied in **education** and **entertainment**.

10. Data Management and Analytics AI excels in processing high volumes of data, extracting insights, and improving data reliability.

- **Data Observability and Reliability:** **Monte Carlo's** data observability platform uses **machine learning-powered anomaly detection** to help organizations improve data reliability, quickly identify issues, and streamline resolutions.
- **Legal Research and Analysis:** **Harvey AI** is a generative platform for legal professionals, trained to process legal jargon, analyze contracts, ensure compliance, assist with litigation, and provide research assistance by accessing cases and precedent. **Hebbia AI** also serves the legal sector by allowing custom AI agents to extract, structure, and analyze large numbers of documents.
- **Text and Document Analysis:** **Kensho Technologies** uses **natural language processing (NLP)** and **machine learning** to create AI applications, including its Classify product which analyzes text and documents for research, delivering research-quality annotations. **Accuris's** AI-powered Engineering Workbench Professional offers **NLP, machine learning, semantic research, document understanding, and information retrieval** to help users find knowledge in engineering workflows.
- **Data Exchange and Transformation:** **Flatfile's** data exchange platform features an AI-intelligent Transform feature for accurate and fast bulk edits to large data files using **natural language commands**.
- **User Research and Insights:** **Dscout** provides a human-centered research platform where AI analysis transforms user research data into actionable insights.
- **Data Science Development:** **Domino Data Lab** helps enterprises expedite data science development and deployment, providing tools for building and productizing **generative AI**, including model fine-tuning and prompt engineering.

11. Public Safety and Security AI contributes to safety and security operations, including fraud prevention and content protection.

- **Cybersecurity and Fraud Prevention:** AI enables cybersecurity companies to deploy faster solutions. **Riskified** uses AI for fraud detection in e-commerce. **Redflag AI's** content protection platform uses AI to find instances of unauthorized content use on the

web, checking for piracy, fraud, copyright infringement, and cybersecurity issues. **Greenlight Guru's** AI search engine detects and assesses security risks in network devices. **Cash App** uses **machine learning and AI** for risk mitigation and fraud prevention. **Visual surveillance** and **human identification** using biometrics (face, fingerprint, iris, hand) are also applications.

12. Creative and Content Generation Advances in **large language models** and **generative AI** have led to powerful AI tools for content creation.

- **Text and Content Creation:** **OpenAI's ChatGPT** acts as a collaborative partner for coding tasks and can process written text, audio, images, and videos. **Anthropic's Claude chatbot** generates human-like responses, proofreads text, writes song lyrics, creates recipes, and builds software applications. **Notion** provides AI tools for text generation.
- **Image and Video Generation:** **Hedra** offers an AI solution that develops talking character videos from images, text, or voice inputs, with features like aspect ratio adjustments and voice cloning.
- **Captioning:** **Mixbook** features a **generative AI tool** that helps with caption writing for photo products, analyzing uploaded images and producing relevant options.
- **Language Translation:** **Smartcat** is an AI platform that converts content (videos, websites, software) into any language, boasting multilingual enterprise library capabilities.
- **Search Engines:** **Perplexity** is an AI-powered search engine that answers questions through a conversational interface, interpreting user prompts and delivering direct, plain language responses to factual, complex, and coding inquiries.

13. Specialized Industries

- **Aerospace and Space Operations:** **The Aerospace Corporation** envisions AI integrated into space operations across areas like post-mission analysis, execution, and mission planning.
- **Sports Analytics:** **Genius Sports** offers data analytics solutions to sports teams, with an AI-powered video platform trained to understand contextual clues from live gameplay and an AI tool to locate player body contours for close calls.

- **Real Estate Marketing:** Digible's Fiona, an AI-powered virtual assistant, helps users build data-driven marketing and budget plans for apartment marketing and monitors for Fair Housing violations.

1.4.

1. Types of AI (Based on Functionality)

When explaining different AI systems, they can also be categorized based on their functionalities:

- **Reactive Machines AI**
 - **Definition:** This type of AI operates solely based on present data, considering only the current situation.
 - **Characteristics:** Reactive AI machines cannot form inferences from past data or evaluate future actions based on memory. They are limited to performing a narrow range of pre-defined tasks.
 - **Example:** A notable example is IBM's Deep Blue, the chess program that defeated world champion Garry Kasparov.
- **Limited Memory AI**
 - **Definition:** Limited Memory AI systems can make informed and improved decisions by studying past data stored in their temporary or short-lived memory.
 - **Characteristics:** This memory allows them to store past experiences and evaluate future actions.
 - **Examples:** Most present-day AI applications, including self-driving cars, chatbots, and virtual assistants, fall into this category. Self-driving cars, for instance, use recent data from sensors to identify pedestrians, steep roads, and traffic signals to make immediate driving decisions and prevent accidents.
- **Theory of Mind AI**
 - **Definition:** This is a more advanced type of Artificial Intelligence that is still under development.
 - **Characteristics:** This category of machines is speculated to play a significant role in psychology. It will focus mainly on **emotional intelligence** so that human

beliefs and thoughts can be better comprehended. This type requires the machine to truly understand humans.

- **Current Status:** Rigorous research is ongoing in this area, but it has not yet been fully developed.

- **Self-Aware AI**

- **Definition:** This represents the future of AI where machines will possess their own consciousness, sentience, and self-awareness.
- **Characteristics:** This type of AI would be capable of understanding and possessing emotions, which could lead them to form beliefs and desires.
- **Current Status:** Given current circumstances, this type of AI is considered "far-fetched" or still theoretical. As mentioned earlier, geniuses like Elon Musk and Stephen Hawking have warned about the potential implications of AI reaching this superintelligence stage.

2. Branches of Artificial Intelligence (Based on Technologies/Techniques)

AI is a vast field encompassing various domains, processes, and techniques that are implemented to solve real-world problems. These branches include:

- **Machine Learning (ML)**

- **Definition:** ML is the science of enabling machines to interpret, process, and analyze data to solve real-world problems. It involves developing algorithms that can learn from data without direct programming.
- **Categories:** Machine Learning is broadly categorized into three types: **Supervised Learning, Unsupervised Learning, and Reinforcement Learning.**
- **Applications:** ML algorithms are used in various applications, including image recognition, spam filtering, and natural language processing.

- **Deep Learning**

- **Definition:** Deep Learning is an advanced field of Machine Learning that involves implementing **Neural Networks** on high-dimensional data to gain insights and form solutions. It harnesses artificial neural networks to acquire knowledge from data.

- **Applications:** Deep Learning is the logic behind technologies like Facebook's face verification algorithm, self-driving cars, and virtual assistants such as Siri and Alexa. It is effective for problems like voice control, image recognition, and speech recognition.
- **Natural Language Processing (NLP)**
 - **Definition:** NLP is the science of deriving insights from natural human language to enable communication with machines and support business growth. It deals with the interaction between computers and human language.
 - **Applications:** NLP is used by platforms like Twitter to filter out terroristic language in tweets and by Amazon to understand customer reviews and improve user experience. Other applications include chatbots, translation services, sentiment analysis, machine translation, speech recognition, and text analysis.
- **Robotics**
 - **Definition:** Robotics is a branch of Artificial Intelligence that focuses on the design, construction, operation, and application of robots. AI Robots are artificial agents that act in real-world environments to produce results by taking accountable actions.
 - **Applications:** AI enhances robot capabilities through **computer vision** (enabling robots to detect objects and recognize images, videos, or human beings). It also utilizes **edge computing** for processing large amounts of data before cloud storage. Robots with AI are used as chatbots in customer service and for packaging purposes in many industries. Sophia the humanoid is a prominent example of AI in robotics.
- **Expert Systems**
 - **Definition:** An expert system is an AI-based computer system designed to learn and reciprocate the decision-making ability of a human expert. Unlike human experts, AI decisions are not affected by emotions or memory loss, offering high security and accuracy (up to 99% with correct input data).
 - **Characteristics:** Expert systems use **if-then logical notations** to solve complex problems and do not rely on conventional procedural programming. They answer questions and solve problems in specific domains using rule-based systems.
 - **Applications:** They are widely used in the financial sector to identify fraud or suspicious activity. In medical fields, they help detect diseases like cancer (e.g.,

CaDet), identify bacteria (e.g., MYCIN), and perform chemical analysis (e.g., DENDRAL). They also assist students in educational sectors, and are used in information management, loan analysis, and virus detection.

- **Computer Vision**

- **Definition:** Computer vision is a branch of computer science focused on developing systems capable of processing, interpreting, and perceiving pictures/videos similar to human vision. Unlike human eyes, optic nerves, and visual cortex, computers use cameras, data, and algorithms for vision. This technology allows machines to interpret the world visually.
- **Applications:** It is used for **object recognition** (identifying, locating, and categorizing objects, such as traffic signals or faces in a crowd). It also aids in **facial recognition**, recognizing patterns and edges in images, and detecting weed growth, nutrient deficits, or diseases in plants/crops in **agriculture**. Other applications include medical image analysis, surveillance, and manufacturing.

- **Fuzzy Logic**

- **Definition:** Fuzzy logic is a computing approach based on the principles of "degrees of truth," diverging from the traditional boolean (true/false) logic of modern computers.
- **Applications:** It is employed in medical fields to solve complex decision-making problems, as well as in automatic gearboxes and vehicle environment control systems.

1.5.

Figure 7 provides an illustrative hierarchy of various types of intelligent systems, categorizing them into several domains:

- **Autonomous Robots:** These are representative intelligent systems that typically include most of the functions described in the paper.
- **Management Support Systems:** These systems assist human operators in making decisions for complex, dynamic environments.
- **First Generation Expert Systems:** These systems, popular in the 1980s, primarily used heuristic reasoning with symbolic knowledge representations.
- **Recommendation Systems:** These systems recommend economic investments.

- **Natural Language Interfaces:**
- **Search Engines:**
- **Video Games:**
- **Computer Vision Systems:**
- **Machine Translation Systems:**
- **Robotic Process Automation (RPA):**
- **Automated Trading Systems:**
- **Knowledge-Based Systems:**
- **Intelligent Tutoring Systems:**
- **Decision Support Systems:**

The paper elaborates on some of these categories, providing concrete examples:

1. Autonomous Robots

Autonomous robots are a prominent example of intelligent systems, often embodying a majority of the functional components described in the paper. This category encompasses systems with varying degrees of complexity, including industrial collaborative robots, autonomous vehicles (like self-driving cars and planetary explorers), assistive robots, domestic robots (e.g., house cleaning robots), and entertainment robots.

An example is an **aerial robot developed for airplane inspection** in 2019 by Airbus. This demonstrative prototype's goal is to assist maintenance engineers in inspecting aircraft surfaces for defects. The robot flies close to the aircraft and processes images from on-board cameras. Its functional organization, following the paper's approach, includes:

- **Environment data acquisition:** The aerial robot is equipped with multiple sensors:
 - **Inertial Measurement Unit (IMU):** Provides data on orientation, angular velocities, and linear accelerations.
 - **Cameras:** Used for velocity calculation via visual odometry.
 - **Lidar sensor:** Generates a 3D point cloud.
 - **High-resolution frontal camera:** For detailed imaging.
- **World modeling:** The system constructs a 3D representation of the world and localizes its position by fusing data from various sensors using an Extended Kalman Filter (EKF).

- **Situation analysis:** The high-resolution camera identifies anomalies on the airplane surface using computer vision, spatially correlating them with the robot's position.
- **Reactive decisions:** The system incorporates reactive mechanisms for immediate responses:
 - **Obstacle avoidance:** Stops the vehicle upon detecting unexpected obstacles via the lidar sensor.
 - **Safety landing:** Initiates an immediate landing if the battery charge falls too low.
- **Reasoning about actions:** The robot generates an inspection plan (via path planning) to ensure comprehensive coverage of the airplane's surface.
- **Execution control:** It uses motion control mechanisms to carry out the navigation plan, specifically employing a motion controller with a path tracking algorithm and PID control.
- **Execution monitoring:** This function supervises motion control to detect the completion of partial goals and attempts to verify the correct execution of actions.
- **Communicating with others:** The operator inputs the airplane's geometry and relevant inspection locations. The operator can also intervene to stop the mission in emergencies.

2. First Generation Expert Systems

First-generation expert systems were instrumental in the success of AI in the 1980s, leading to many commercial applications across various domains. These systems primarily utilized **heuristic reasoning** with **symbolic knowledge representations**, such as rules.

A foundational example is **Mycin**, developed at Stanford University in the early 1970s, which served as a model for subsequent systems. Mycin's primary function was to **diagnose infectious diseases**. Its functional components included:

- **Communicating with others:** Information about patients was gathered through a **question-answer dialogue** with the user, offering predefined sets of potential answers. Mycin asked questions about the patient and cultures, allowing users to respond with uncertainty and request explanations. It presented diagnostic results and recommended therapies.
- **Situation analysis:** Mycin diagnosed the causes of symptoms by identifying possible infectious organisms. It employed a **heuristic classification method** with a knowledge

base containing approximately 450 rules and used **backward chaining**. Notably, Mycin was one of the first systems capable of generating explanations, presenting the rules used during reasoning to justify its conclusions. It also featured an original method for approximate reasoning using certainty factors.

- **Reasoning about actions:** Mycin recommended therapies for the identified infectious organisms using a **generate and test procedure**.

Despite their success, these systems faced limitations, primarily the **knowledge acquisition bottleneck**, where manually creating the content of large knowledge bases (e.g., rules) required significant effort. Another limitation was their **rigid data acquisition mechanism**, relying on predefined answers, which restricted their integration into day-to-day operations compared to modern sensor-based or unstructured text processing methods.

3. Management Support Systems

Modern management support systems have emerged in strategic areas like smart cities (e.g., transportation, climate, pollution monitoring) and Industry 4.0 (sensorized industrial installations). These systems are built upon networks of sensors that monitor complex, dynamic environments and are designed to assist operators in making management decisions. Examples include expert systems for public transportation management, airport scheduling and coordination, emergency decision support in floods, and electric power operation.

An illustrative example is an **expert system for urban bus transportation management** included in the fleet management system of Vitoria, Spain's bus control center in 2002. This system detects incidents and recommends responses within the urban bus network. Its functional organization includes:

- **Environment data acquisition:** The system collects real-time data on the spatial location of bus fleets using **GPS sensors**.
- **World modeling:** It utilizes a map that characterizes transport lines (stops, distances, transfer points) and tracks each bus's position on its corresponding line. The model also includes information on reserve buses and maintenance vehicles.
- **Situation analysis:** The system identifies incidents, such as **delays**, by comparing current and planned bus positions. It also **predicts short-term bus passenger demand** based on current time, date, and historical patterns.
- **Assigning value to the situation:** It evaluates the severity of detected delays on each line by factoring in the predicted bus passenger demand.

- **Reasoning about actions:** The system determines actions for incident management, employing an **automated planner based on Hierarchical Task Network (HTN) planning** to generate action plans.
- **Communicating with others:** The system presents detected incidents (e.g., bus delays) to the operator. The operator can also input events not automatically detected by sensors (e.g., broken buses, blocked streets). Crucially, the system recommends actions (e.g., deploying additional buses, sending repair trucks) and **justifies these recommendations**, for instance, by showing prioritization of lines based on expected demand.

In such management support systems, while the system provides recommendations and evaluations, the **human operator retains responsibility for making the final management decisions and implementing actions** in the environment. Therefore, the system's ability to communicate justified recommendations is critical for building operator trust and accountability.

1.6.

1. NumPy, short for Numerical Python, is the **fundamental package for numerical computation in Python**. Its core contribution is the **n-dimensional array (ndarray)**, a powerful data structure that serves as the foundation for nearly all Python's data science toolkit.

Purpose and Features:

- **Efficiency and Speed:** NumPy operations are implemented in C, enabling calculations that complete in nanoseconds rather than seconds. This efficiency makes it suitable for large-scale numerical tasks.
- **Vectorization:** NumPy facilitates **vectorized operations**, allowing users to perform the same operation on every element in an array without explicit for loops. This leads to **clearer, more concise code** that often mirrors mathematical equations.
- **Core Mathematical Capabilities:** It provides efficient array operations, vectorized mathematical functions, scientific computing functions, and linear algebra operations.
- **Quality and Reliability:** With thousands of contributors, NumPy is continuously maintained to be fast, friendly, and bug-free.

Key Concepts and Operations:

- **Arrays, Shape, and Axes:** NumPy arrays have a `.shape` property, which is a tuple indicating the size of each dimension. Operations can be applied across specific **axes** (dimensions), which are zero-indexed (e.g., axis 0 for vertical, axis 1 for horizontal in 2D arrays).
- **Broadcasting:** This mechanism allows NumPy to perform operations on arrays of different shapes by implicitly duplicating values along dimensions where sizes differ or where one array has a dimension size of 1.
- **Data Types:** NumPy requires all items in an array to be of the same underlying C type for optimization, supporting various numerical types (integers, booleans, floats, complex numbers) with different bit sizes, as well as sized Unicode strings. It also features **structured arrays (record arrays)**, which allow for column-specific types and names, resembling tabular data.
- **Data Manipulation:** NumPy offers methods for **indexing** (accessing elements by position), **masking and filtering** (selecting elements based on Boolean conditions), **transposing** (switching rows and columns), **sorting** (arranging elements), and **concatenating** (joining arrays).
- **Aggregation:** Functions like `.sum()`, `.max()`, `.mean()`, and `.std()` can aggregate data across entire arrays or along specific axes.

Illustrated Examples:

- **Curving Test Grades:** To apply a curve to an array of student grades, NumPy's **vectorization** and **broadcasting** allow for concise code:

```
[ ]: # Nguyễn Hoàng Tùng

[5]: import numpy as np
      grades = np.array([ 91.25, 54.25, 83.25, 100. , 70.25, 100. , 93.25, 31.25])
      average = grades.mean()
      change = 80 - average # CURVE_CENTER - average
      new_grades = grades + change # Vectorized addition
      curved_grades = np.clip(new_grades, grades, 100) # Broadcasting for min/max
      print(curved_grades)

[ 93.3125  56.3125  85.3125 100.         72.3125 100.         95.3125  33.3125]
```

This example demonstrates how a scalar change is added to each grade element (broadcasting), and how `np.clip()` limits values within a range, also using broadcasting.

- **Image Manipulation:** Images can be represented as NumPy arrays (e.g., a width × height × 3 array for RGB channels). Operations like converting an image to grayscale involve mathematical transformations on these arrays, leveraging **dot products** for weighted averages:

```
[ ]: # Nguyễn Hoàng Tùng

[ ]: # Example concept (actual code uses matplotlib.image)
img = mpimg.imread("kitty.jpg") # Assumed NumPy array
weights = np.array([0.3, 0.59, 0.11])
grayscale = img @ weights # Dot product for weighted average
mpimg.imsave("good-gray.jpg", grayscale, cmap="gray")
```

This illustrates how powerful numerical operations on arrays can be applied to real-world data like image pixels.

2. Pandas is a library specifically designed for **working with spreadsheet or tabular data**. It serves as the primary tool for initial data handling, exploration, cleaning, and transformation in data science workflows.

Purpose and Features:

- **Core Data Structures:** Pandas offers two primary data structures:
 - **Series:** A **one-dimensional labeled array** capable of holding various data types.
 - **DataFrame:** A **two-dimensional, table-like data structure** with labeled rows (index) and columns.
- **Data Import/Export:** Pandas excels at reading data from and writing data to various sources, including CSV, Excel, and Parquet files, making it highly versatile for real-world data handling.
- **Data Exploration and Summarization:** It provides methods to quickly inspect data, such as `head()` and `tail()` to view top/bottom rows, `dtypes` to check column data types, `info()` for a summary of the DataFrame, `columns` and `index` to view labels, and `describe()` for statistical summaries.
- **Data Cleaning and Missing Values:** Pandas facilitates handling missing data (represented by `np.nan`), with functions like `dropna()` to remove rows with missing values, `fillna()` to replace them, and `isna()` to identify them.
- **Data Transformation:** It supports a wide range of transformations, including sorting by index (`sort_index()`) or values (`sort_values()`), applying custom functions (`apply()`, `transform()`, `agg()`), and string processing methods via the `.str` attribute.
- **Combining Data:** Pandas provides robust capabilities for combining data, such as `concat()` for stacking objects row-wise, and `merge()` for SQL-style joins based on shared keys.

- **Grouping and Reshaping:** The `groupby()` method allows for splitting data into groups, applying functions (like `sum()`, `mean()`), and combining results. `pivot_table()` enables reshaping data for summarization, while `stack()` and `unstack()` compress or uncompress column levels.

Data Access and Manipulation:

- **Column Selection:** Columns can be selected using single labels (e.g., `df['A']`) which yields a Series.
- **Row Slicing:** Rows can be selected using integer or label-based slicing (e.g., `df[0:3]` or `df["20130102":"20130104"]`).
- **Label-based Selection (.loc):** Used for selecting data by explicit row and column labels, including slices where both endpoints are included.
- **Position-based Selection (.iloc):** Used for selection by integer positions, similar to NumPy/Python array indexing.
- **Scalar Access (.at, .iat):** Optimized methods for fast access to a single scalar value by label (.at) or position (.iat).
- **Boolean Indexing:** Data can be filtered based on conditions that return a boolean Series or DataFrame. The `isin()` method is useful for filtering based on a list of values.
- **Setting Values:** New columns can be added, and existing values can be modified by label (.at), by position (.iat), or by assigning NumPy arrays or series.

Relationship with NumPy:

Pandas is built on top of NumPy. Pandas DataFrames can be readily converted to NumPy arrays using the `.values` attribute, which is crucial for integrating with libraries that primarily accept NumPy inputs, like scikit-learn. Pandas 2.x versions incorporate the **PyArrow backend** for enhanced performance, particularly for string operations, offering optimizations over the traditional NumPy backend.

Illustrated Examples:

- **Creating a DataFrame:**

```
[ ]: # Nguyễn Hoàng Tùng
```

```
[6]: import pandas as pd
import numpy as np

dates = pd.date_range("20130101", periods=6)
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
print(df)
```

	A	B	C	D
2013-01-01	0.275366	-1.714427	-0.833081	0.094099
2013-01-02	-0.800041	-0.640832	0.010274	-0.205125
2013-01-03	0.692742	-0.246443	-1.138058	-0.728785
2013-01-04	-1.350872	0.759481	-1.853930	-0.081549
2013-01-05	0.620015	-0.245393	-0.373551	-0.340923
2013-01-06	-1.069233	-0.421430	1.491942	-0.388174

This creates a DataFrame df with a datetime index and labeled columns.

- **Data Filtering with Boolean Indexing:**

```
[ ]: # Nguyễn Hoàng Tùng
```

```
[6]: import pandas as pd
import numpy as np

dates = pd.date_range("20130101", periods=6)
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
print(df)
```

	A	B	C	D
2013-01-01	0.275366	-1.714427	-0.833081	0.094099
2013-01-02	-0.800041	-0.640832	0.010274	-0.205125
2013-01-03	0.692742	-0.246443	-1.138058	-0.728785
2013-01-04	-1.350872	0.759481	-1.853930	-0.081549
2013-01-05	0.620015	-0.245393	-0.373551	-0.340923
2013-01-06	-1.069233	-0.421430	1.491942	-0.388174

```
[7]: # Assume df is the DataFrame created above
df2 = df.copy()
df2["E"] = ["one", "one", "two", "three", "four", "three"]
filtered_df = df2[df2["E"].isin(["two", "four"])]
print(filtered_df)
```

	A	B	C	D	E
2013-01-03	0.692742	-0.246443	-1.138058	-0.728785	two
2013-01-05	0.620015	-0.245393	-0.373551	-0.340923	four

This example demonstrates filtering rows where column "E" contains specific values using the `isin()` method.

- **Grouping Data:**

```
[ ]: # Nguyễn Hoàng Tùng

[6]: import pandas as pd
import numpy as np

dates = pd.date_range("20130101", periods=6)
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
print(df)
```

```
      A         B         C         D
2013-01-01  0.275366 -1.714427 -0.833081  0.094099
2013-01-02 -0.800041 -0.640832  0.010274 -0.205125
2013-01-03  0.692742 -0.246443 -1.138058 -0.728785
2013-01-04 -1.350872  0.759481 -1.853930 -0.081549
2013-01-05  0.620015 -0.245393 -0.373551 -0.340923
2013-01-06 -1.069233 -0.421430  1.491942 -0.388174
```

```
[7]: # Assume df is the DataFrame created above
df2 = df.copy()
df2["E"] = ["one", "one", "two", "three", "four", "three"]
filtered_df = df2[df2["E"].isin(["two", "four"])]
print(filtered_df)
```

```
      A         B         C         D     E
2013-01-03  0.692742 -0.246443 -1.138058 -0.728785  two
2013-01-05  0.620015 -0.245393 -0.373551 -0.340923  four
```

```
[8]: df_group = pd.DataFrame({
    "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],
    "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
    "C": np.random.randn(8),
    "D": np.random.randn(8),
})
grouped_sum = df_group.groupby("A")[["C", "D"]].sum()
print(grouped_sum)
```

```
      C         D
bar  0.782486 -0.10169
foo -0.471164  6.06116
```

This shows grouping by column "A" and then summing columns "C" and "D" for each group.

3. Matplotlib is a comprehensive Python library for **creating static, animated, and interactive visualizations** in Python. It is commonly used to communicate data insights to other humans.

Purpose and Features:

- **Versatile Plotting:** Matplotlib provides a straightforward interface to create a wide variety of charts, including line plots, scatter plots, and bar charts.
- **Customization:** It allows for extensive customization of plots, including adjusting colors, markers, line styles, titles, labels, legends, and grid lines, to suit presentation needs.
- **Figure Saving:** Plots can be saved to various file formats (e.g., PNG) with specified resolution for reports or later use.
- **Integration:** It is often used in conjunction with NumPy for generating sample data for plots and with Pandas, as DataFrames have a built-in `.plot()` method that leverages Matplotlib.

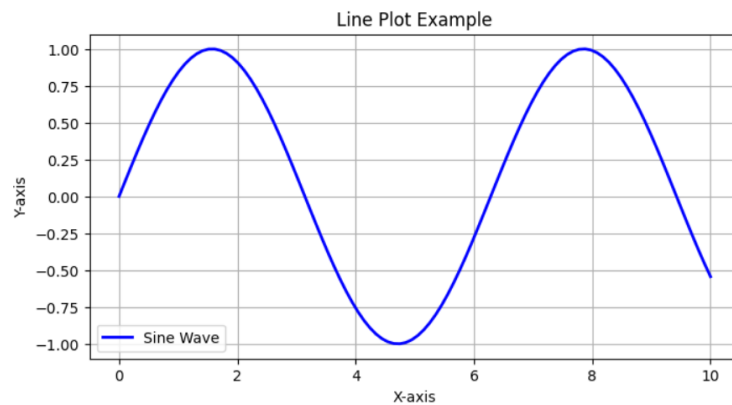
Illustrated Examples:

- **Creating a Line Plot:**

```
[ ]: # Nguyễn Hoàng Tùng
```

```
[9]: import matplotlib.pyplot as plt
import numpy as np

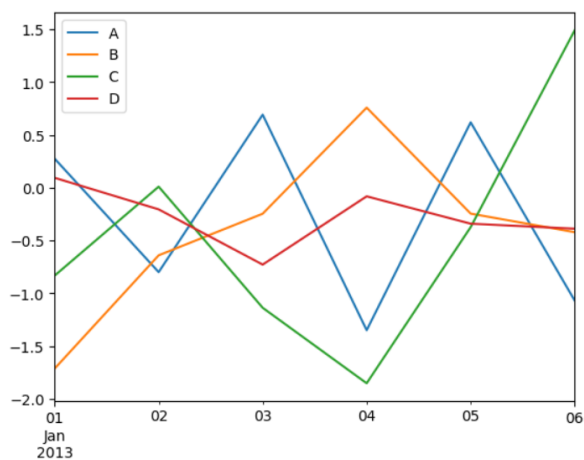
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.figure(figsize=(8, 4))
plt.plot(x, y, label="Sine Wave", color="blue", linewidth=2)
plt.title("Line Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)
plt.show()
```



This code generates and displays a simple sine wave line plot with various customizations.

- **Plotting DataFrame Columns:**

```
[10]: # Assume df is a pandas DataFrame
df.plot();
plt.legend(loc='best');
plt.show() # Not explicitly shown, but typically follows
```



This demonstrates how Matplotlib can plot all columns of a DataFrame using its built-in `plot()` method.

4. Scikit-learn is a powerful Python library that serves as a **modeling toolkit for machine learning**. It provides a consistent API for various machine learning tasks, from data preprocessing to model building and evaluation.

Purpose and Features:

- **Learning Problems:** Scikit-learn addresses several categories of learning problems:
 - **Supervised Learning:** Where data includes attributes to be predicted. This encompasses:
 - **Classification:** Assigning data samples to discrete categories (e.g., handwritten digit recognition, spam detection).
 - **Regression:** Predicting one or more continuous variables (e.g., house price prediction, salmon length prediction).
 - **Unsupervised Learning:** Where training data lacks target values, focusing on discovering underlying data structures. This includes:
 - **Clustering:** Discovering groups of similar examples within data.
 - **Density Estimation:** Determining the distribution of data.
 - **Dimensionality Reduction:** Projecting high-dimensional data into lower dimensions for visualization or efficiency.
- **Consistent API:** Estimators in scikit-learn (Python objects for classification or regression) typically implement `fit(X, y)` to learn from training data and `predict(T)` to make predictions on new data.
- **Algorithm Variety:** It offers a wide array of machine learning algorithms, including:
 - **Regression:** Linear Regression.
 - **Classification:** Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Naive Bayes, Decision Trees.
 - **Ensemble Methods:** Random Forest and Boosting algorithms (AdaBoost, Gradient Boosting, XGBoost).
 - **Clustering:** K-Means.
 - **Dimensionality Reduction:** Principal Component Analysis (PCA).

- **Preprocessing and Model Evaluation:** Provides tools for consistent data preprocessing and evaluating model performance using various metrics (e.g., Mean Squared Error, R^2 Score).
- **Model Persistence:** Trained models can be saved and reloaded using Python's pickle or joblib for future use.

Illustrated Examples:

- **Learning and Predicting with SVC (Support Vector Classifier):**

```
[ ]: # Nguyễn Hoàng Tùng

•[12]: from sklearn import svm, datasets

# Load example dataset (digits for handwritten digit recognition)
digits = datasets.load_digits()
X, y = digits.data, digits.target

# Create an SVC estimator instance
clf = svm.SVC(gamma=0.001, C=100.)

# Fit the model using training data (all but the last image)
clf.fit(X[:-1], y[:-1])

# Predict the class of an unseen sample (the last image)
prediction = clf.predict(X[-1:])
print(f"Predicted digit: {prediction}")

Predicted digit: [8]
```

This shows the basic fit() and predict() workflow for a classification problem.

- **Training and Evaluating Regression Models:**

```
[ ]: # Nguyễn Hoàng Tùng

[21]: from sklearn.linear_model import LinearRegression
      from sklearn.ensemble import RandomForestRegressor
      from sklearn.metrics import r2_score
      import pandas as pd

      # Create a small sample dataset
      data = {
          "Cement": [540, 540, 332, 332, 198],
          "Slag": [0, 0, 142, 142, 132],
          "FlyAsh": [0, 0, 0, 0, 0],
          "Water": [162, 162, 228, 228, 192],
          "Superplasticizer": [2.5, 2.5, 0, 0, 0],
          "CoarseAggregate": [1040, 1055, 932, 932, 978],
          "FineAggregate": [676, 676, 594, 594, 825],
          "Age": [28, 28, 270, 365, 360],
          "Strength": [79.99, 61.89, 40.27, 41.05, 44.30] # target column
      }

      concrete_data = pd.DataFrame(data)

      # Split into features (X) and target (y)
      X = concrete_data.drop("Strength", axis=1)
      y = concrete_data["Strength"]

      # Train Linear Regression on the full dataset
      lr_model = LinearRegression()
      lr_model.fit(X, y)
      lr_predictions = lr_model.predict(X)

      # Train Random Forest on the full dataset
      rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
      rf_model.fit(X, y)
      rf_predictions = rf_model.predict(X)

      # Evaluate on the same dataset (not real validation!)
      print(f"Linear Regression R² Score: {r2_score(y, lr_predictions):.2f}")
      print(f"Random Forest R² Score: {r2_score(y, rf_predictions):.2f}")

      Linear Regression R² Score: 1.00
      Random Forest R² Score: 0.94
```

This illustrates training and comparing two regression models, Linear Regression and Random Forest, using NumPy arrays as input and evaluating their performance.

5. Seamless Integration in a Data Science Workflow

The true power of these libraries emerges when they are combined in a **cohesive workflow**.

1. **Pandas** is typically used first to **load and prepare data** from various sources, handling initial exploration, cleaning, and reshaping.
2. The cleaned Pandas DataFrames are then **converted into NumPy arrays** (e.g., using `.values`), which serve as the standard input format for efficient numerical computations and machine learning algorithms. NumPy's vectorized operations are also leveraged for **feature engineering** by creating new features from existing ones.
3. Finally, **scikit-learn** utilizes these NumPy arrays to **build, train, and evaluate machine learning models**.
4. Throughout the process, **Matplotlib** is indispensable for **visualizing data insights** during exploration and presenting model results effectively.

This integrated approach allows data scientists to move smoothly between data handling, numerical computation, and predictive modeling without friction, fostering quick iteration from raw data to accurate predictions.

1.7.

```
[1]: # Nguyễn Hoàng Tùng

import numpy as np

# Define the arrays for names, heights, and weights as specified in the assignment [2]
names = np.array(['Ann', 'Joe', 'Mark'])
heights = np.array([1.5, 1.78, 1.6]) # Heights in meters [2]
weights = np.array([65, 46, 59]) # Weights in kgs [2]

# Calculate the Body Mass Index (BMI) using the formula: BMI = weight / (height^2) [1, 2]
# NumPy allows for element-wise operations on arrays, making this calculation straightforward [1]
bmi = weights / heights ** 2

# Print the calculated BMI for each individual [1]
print("Calculated BMI values:", bmi)

# Further, you can use Boolean indexing to identify individuals based on their BMI,
# as demonstrated in the source [1]:
print("\nBMI Categories:")
print("Overweight: ", names[bmi > 25]) # Individuals with BMI > 25 [1]
print("Underweight: ", names[bmi < 18.5]) # Individuals with BMI < 18.5 [1]
print("Healthy: ", names[(bmi >= 18.5) & (bmi <= 25)]) # Individuals with BMI between 18.5 and 25 (inclusive) [1]

Calculated BMI values: [28.88888889 14.51836889 23.046875 ]

BMI Categories:
Overweight: ['Ann']
Underweight: ['Joe']
Healthy: ['Mark']
```

1.8.

```
[2]: # Nguyễn Hoàng Tùng

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd # Used for better tabular data display

# 1. Generate 4 sets of student names, 5 subject names, and grades on a scale of 4
student_names = np.array(['Nguyễn Hoàng Tùng', 'Đinh Quyết Thắng', 'Trần Thành Trung', 'Tường Duy Minh Tú'])
subjects = np.array([
    'Introduction to Software Engineering',
    'Introduction to Artificial Intelligent',
    'Web Development',
    'Database Management System',
    'Internship'
])

num_students = len(student_names)
num_subjects = len(subjects)

# Generate random grades (scale of 0.0 to 4.0) for each student in each subject
# np.random.uniform generates random floats within a specified range.
# grades array will have shape (num_students, num_subjects)
grades = np.random.uniform(0.0, 4.0, size=(num_students, num_subjects))

# Calculate "Intelligent System Development" score (average of the 5 subjects) for each student
# This serves as the consolidated result named 'Intelligent System Development'.
# The 'axis=1' argument means the mean is calculated across the second dimension (subjects).
intelligent_system_development_scores = np.mean(grades, axis=1)
```

```

# Display the generated tabular data for clarity
print("\nGenerated Student Grades (on a scale of 4.0):")
grades_df = pd.DataFrame(grades, index=student_names, columns=subjects)
print(grades_df.round(2)) # Display grades rounded to 2 decimal places

print("\nIntelligent System Development Scores (Average Grade):")
for i in range(num_students):
    print(f"- {student_names[i]}: {intelligent_system_development_scores[i]:.2f}")

# 2. Display forms corresponding to page 71-73 of book 1

# --- Plotting Multiple Lines in the Same Chart (pages 71-72, book 1) --- [1, 2]
# This chart shows each student's performance across the 5 subjects.
plt.figure(figsize=(12, 7)) # Adjust figure size for better readability

# Loop through each student to plot their grades as a separate line
for i in range(num_students):
    # plt.plot() is used to plot a line chart [1]
    # The 'label' argument is used for the legend [2]
    plt.plot(subjects, grades[i], markers='o', linestyle='-', label=student_names[i])

# Adding a title and labels for the axes [4]
plt.title('Student Performance Across Different Subjects', fontsize=14)
plt.xlabel('Subjects', fontsize=12)
plt.ylabel('Grade (Scale of 4.0)', fontsize=12)
plt.ylim(0, 4.2) # Set y-axis limits to clearly show grades on a scale of 4.0
plt.xticks(rotation=45, ha='right', fontsize=10) # Rotate x-axis labels for readability [5]
plt.yticks(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6) # Add a grid for better data interpretation
plt.legend(title='Student', bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=10, title_fontsize=12) # Adding a legend to distinguish lines [2, 6]
plt.tight_layout() # Adjust layout to prevent labels from overlapping
plt.show()

```

```

# --- Plotting Bar Charts (pages 73-77, book 1) --- [3]
# This chart compares the "Intelligent System Development" score (average grade) for each student.
plt.figure(figsize=(10, 6)) # Adjust figure size
bar_width = 0.6 # Width of the bars

# plt.bar() is used to plot a bar chart [3]
bars = plt.bar(student_names, intelligent_system_development_scores, width=bar_width, color=['skyblue', 'lightcoral', 'lightgreen', 'gold'])

# Adding a title and labels for the axes [4]
plt.title('Intelligent System Development Scores per Student', fontsize=14)
plt.xlabel('Student Name', fontsize=12)
plt.ylabel('Average Grade (Scale of 4.0)', fontsize=12)
plt.ylim(0, 4.2) # Set y-axis limits to match the grade scale
plt.xticks(rotation=45, ha='right', fontsize=10) # Rotate x-axis labels if needed
plt.yticks(fontsize=10)
plt.grid(axis='y', linestyle='--', alpha=0.6) # Add horizontal grid lines for readability [7]

# Add value labels on top of the bars for clarity (not explicitly on page 73 but enhances understanding)
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.1, round(yval, 2), ha='center', va='bottom', fontsize=9)

plt.tight_layout() # Adjust layout
plt.show()

```

Generated Student Grades (on a scale of 4.0):

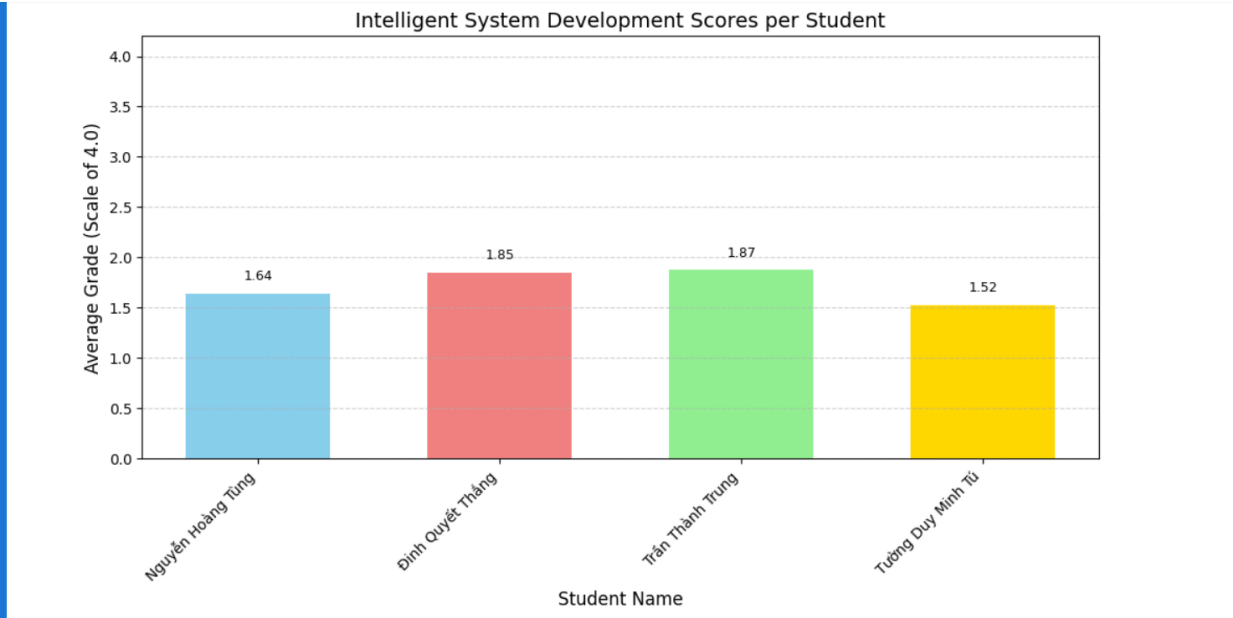
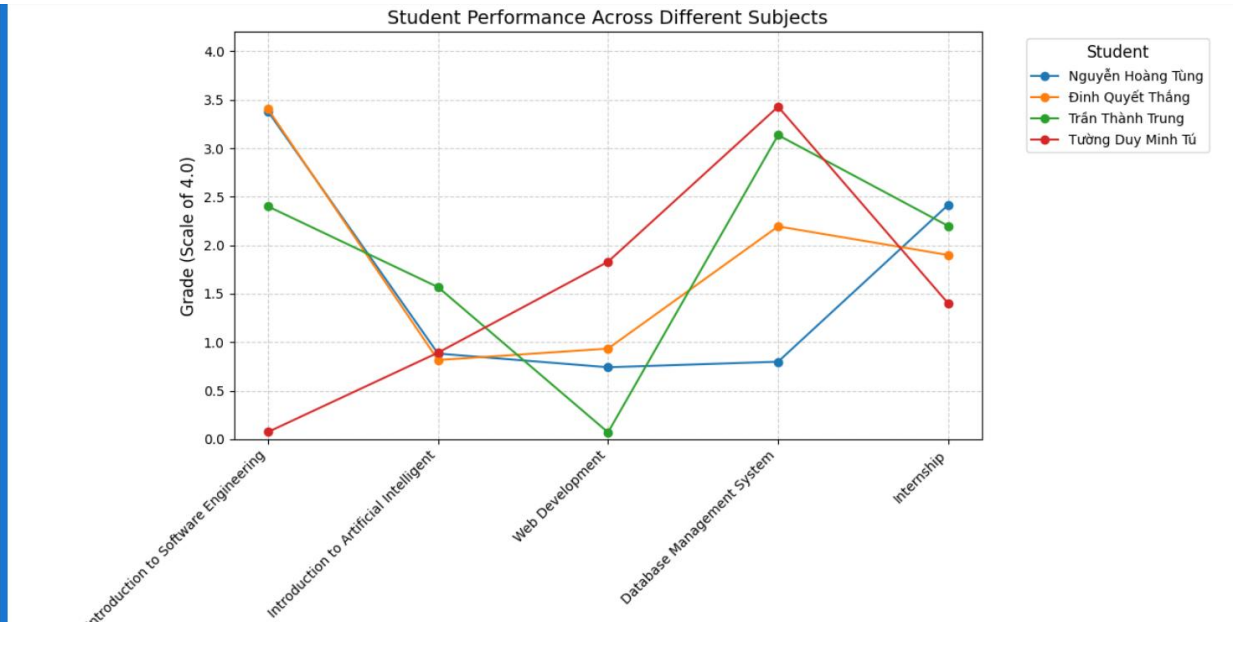
	Introduction to Software Engineering \	
Nguyễn Hoàng Tùng	3.38	
Đinh Quyết Thắng	3.40	
Trần Thành Trung	2.40	
Tường Duy Minh Tú	0.07	

	Introduction to Artificial Intelligent	Web Development \
Nguyễn Hoàng Tùng	0.88	0.74
Đinh Quyết Thắng	0.82	0.93
Trần Thành Trung	1.57	0.07
Tường Duy Minh Tú	0.89	1.83

	Database Management System	Internship
Nguyễn Hoàng Tùng	0.80	2.42
Đinh Quyết Thắng	2.19	1.90
Trần Thành Trung	3.14	2.20
Tường Duy Minh Tú	3.43	1.40

Intelligent System Development Scores (Average Grade):

- Nguyễn Hoàng Tùng: 1.64
- Đinh Quyết Thắng: 1.85
- Trần Thành Trung: 1.87
- Tường Duy Minh Tú: 1.52



1.9.

[10]: # Nguyễn Hoàng Tùng

```
import pandas as pd # Manipulating tabular data
import numpy as np # Numerical operations
import seaborn as sns # Statistical data visualization
import matplotlib.pyplot as plt # Plotting charts

# 1. Generate Synthetic Data
genders = ['Men', 'Women']
groups = ['Group A', 'Group B', 'Group C']
num_samples = 500

# Base random dataset
data = {
    'Gender': np.random.choice(genders, num_samples),
    'Group': np.random.choice(groups, num_samples),
    'Driver_License': np.random.randint(0, 2, num_samples) # 0 = No, 1 = Yes
}
df = pd.DataFrame(data)

# Adjust probabilities for more interesting patterns
# Men in Group A → 75% chance to have a license
mask_men_A = (df['Gender'] == 'Men') & (df['Group'] == 'Group A')
df.loc[mask_men_A, 'Driver_License'] = np.random.choice(
    [0, 1], size=mask_men_A.sum(), p=[0.25, 0.75]
)

# Women in Group C → 30% chance to have a license
mask_women_C = (df['Gender'] == 'Women') & (df['Group'] == 'Group C')
df.loc[mask_women_C, 'Driver_License'] = np.random.choice(
    [0, 1], size=mask_women_C.sum(), p=[0.7, 0.3]
)

# 2. Save & Load Data
csv_filename = "driver_license_data.csv"
df.to_csv(csv_filename, index=False)
print(f"\nSynthetic data with {num_samples} samples saved to '{csv_filename}'")

loaded_df = pd.read_csv(csv_filename)
print("\n--- Sample of Loaded Data from CSV ---")
print(loaded_df.head())

# 3. Show Proportions
print("\n--- Proportion with Driver's License by Group and Gender ---")
print(loaded_df.groupby(['Group', 'Gender'])['Driver_License'].mean().unstack().round(2))

# 4. Plot Visualization
g = sns.catplot(
    x="Gender",
    y="Driver_License",
    col="Group",
    data=loaded_df,
    kind="bar",
    ci=None,
    aspect=0.8,
    palette="viridis"
)
```

```

g.set_axis_labels("Gender", "Proportion with Driver's License")
g.set_titles("Group: {col_name}")
g.fig.suptitle("Proportion of Men and Women with Driver's License by Group", y=1.05, fontsize=16)

plt.tight_layout()
plt.show()

```

```

C:\Users\Admin\AppData\Local\Temp\ipykernel_5868\2619365777.py:24: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an
error in a future version of pandas. Value '[1 0 1 1 1 0 1 0 1 0 0 1 1 1 1 1 0 1 0 1 0 0 1 1 1 1 0 1 1 0 0 1 1 1 1
1 0 1 0 0 1 1 1 0 1 1 1 1 1 0 1 1 1 0 1 1 1 1 1 0 1 1 1 1 0 0 0 1 1 0
0 1 1 1 0 1 0 1 1 1 1 1 1 1 1 0 1 0 1 0 1]' has dtype incompatible with int32, please explicitly cast to a compatible dtype first.
df.loc[mask_men_A, 'Driver_License'] = np.random.choice(

```

Synthetic data with 500 samples saved to 'driver_license_data.csv'

--- Sample of Loaded Data from CSV ---

	Gender	Group	Driver_License
0	Men	Group C	0
1	Women	Group A	1
2	Women	Group B	1
3	Men	Group B	1
4	Women	Group A	0

--- Proportion with Driver's License by Group and Gender ---

Group	Men	Women
Group A	0.68	0.57
Group B	0.57	0.48
Group C	0.53	0.29

```

C:\Users\Admin\AppData\Local\Temp\ipykernel_5868\2619365777.py:48: FutureWarning:

```

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```

g = sns.catplot(
C:\Users\Admin\AppData\Local\Temp\ipykernel_5868\2619365777.py:48: FutureWarning:

```

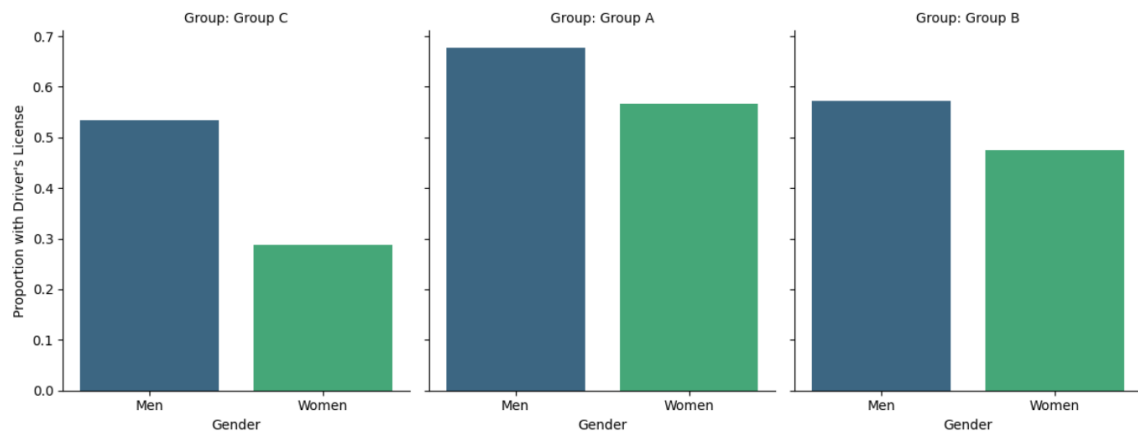
The 'ci' parameter is deprecated. Use 'errorbar=None' for the same effect.

```

g = sns.catplot(

```

Proportion of Men and Women with Driver's License by Group



1.10.

```
# Customize the plot for clarity as shown in the source [1, 12]
g.set_axis_labels("", "Survival Rate") # Set y-axis label, x-axis label is implicitly "who"
g.set_xticklabels(["Men", "Women", "Children"]) # Set x-tick labels for 'who' categories
g.set_titles("{col_name} {col_var}") # Set titles for each subplot, using 'class' as column variable
g.fig.suptitle("Survival Rate of Men, Women, and Children by Class on the Titanic", y=1.02, fontsize=16) # Set a main title for the entire figure

plt.tight_layout() # Adjust layout to prevent labels from overlapping
plt.show() # Display the plot

except FileNotFoundError:
    print(f"Error: The file '{local_file_path}' was not found. Please ensure the path is correct and the file exists.")
except pd.errors.EmptyDataError:
    print(f"Error: The file '{local_file_path}' is empty.")
except pd.errors.ParserError:
    print(f"Error: Could not parse the file '{local_file_path}'. Please check the file format.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Titanic dataset loaded with 891 samples.

C:\Users\Admin\AppData\Local\Temp\ipykernel_9552\2096253389.py:22: FutureWarning:

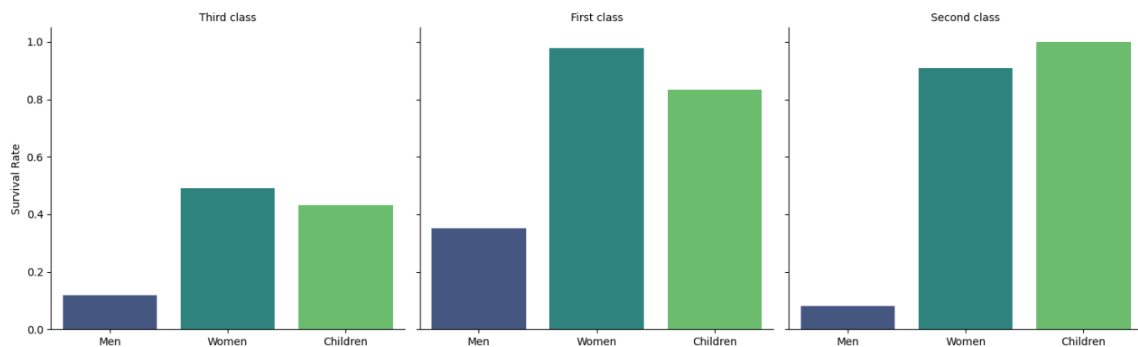
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
g = sns.catplot(
C:\Users\Admin\AppData\Local\Temp\ipykernel_9552\2096253389.py:22: FutureWarning:
```

The 'ci' parameter is deprecated. Use 'errorbar=None' for the same effect.

```
g = sns.catplot(
```

Survival Rate of Men, Women, and Children by Class on the Titanic



1.11.

```
[3]: # Nguyễn Hoàng Tùng

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import os

# Define the filename for the salary dataset
file_name = "salary.csv"
num_entries_per_gender = 500 # Total entries will be 1000

# 1. Generate sample data for salary.csv with 1000 entries
# Create a DataFrame with 'gender' and 'salary' columns as specified in the task [3]
# We'll create a hypothetical dataset with a mix of salaries for men and women
# using numpy.random.normal to simulate realistic distributions.
data = {
    'gender': (['men'] * num_entries_per_gender) + (['women'] * num_entries_per_gender),
    'salary': np.concatenate([
        # Men's salaries centered around 100,000 with a standard deviation of 15,000
        np.random.normal(100000, 15000, num_entries_per_gender),
        # Women's salaries centered around 90,000 with a standard deviation of 12,000
        np.random.normal(90000, 12000, num_entries_per_gender)
    ]).astype(int) # Convert salaries to integers for realism
}

# Ensure no negative salaries due to normal distribution randomness by setting a floor
data['salary'][data['salary'] < 20000] = 20000

salary_df = pd.DataFrame(data)

# Save the DataFrame to a CSV file
try:
    salary_df.to_csv(file_name, index=False)
    print(f"\n--- Sample '{file_name}' with {len(salary_df)} entries generated successfully ---")
    print("First 5 rows of generated data:")
    print(salary_df.head()) # Display the first few rows of the generated data
    print("\nLast 5 rows of generated data:")
    print(salary_df.tail()) # Display the last few rows of the generated data

# 2. Load the data and plot the salary distribution using Seaborn's swarmplot
# The task explicitly refers to page 90 of book 2 for 'sns.swarmplot' [1, 2].
loaded_data = pd.read_csv(file_name)

# Initialize Seaborn style for better aesthetics, as mentioned in book 2 examples
sns.set_style("whitegrid") # This specific style is used in the book's example for swarmplot [1]

# Plotting the swarmplot as demonstrated on page 90 of book 2 [1]
plt.figure(figsize=(10, 7)) # Adjust figure size for better readability
sns.swarmplot(
    x="gender",          # Categorical variable for the x-axis ('men', 'women')
    y="salary",          # Quantitative variable for the y-axis (salary values)
    data=loaded_data,    # The DataFrame containing the salary data
    palette="coolwarm",  # A different color palette for variety, if desired
    size=4              # Adjust marker size for 1000 points
)

plt.title("Salary distribution (1000 entries)", fontsize=16) # Set the title for the plot, as shown in book 2 [1]
plt.xlabel("Gender", fontsize=12) # Label for the x-axis
plt.ylabel("Salary", fontsize=12) # Label for the y-axis
plt.grid(axis='y', linestyle='--', alpha=0.7) # Add a grid for better readability
plt.tight_layout() # Adjust layout to prevent labels from overlapping
plt.show() # Display the plot
```

```
except IOError as e:
    print(f"Error: Could not save or load the file '{file_name}'. {e}")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

--- Sample 'salary.csv' with 1000 entries generated successfully ---
First 5 rows of generated data:

	gender	salary
0	men	143903
1	men	109535
2	men	105454
3	men	83468
4	men	125975

Last 5 rows of generated data:

	gender	salary
995	women	98451
996	women	90123
997	women	71620
998	women	94073
999	women	91553

C:\Users\Admin\AppData\Local\Temp\ipykernel_14884\3680626380.py:49: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.swarmplot(
```



1.12.

```
[2]: # Nguyễn Hoàng Tùng

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. Data (diện tích m², giá nhà tỷ)
X = np.array([50, 60, 65, 70, 75, 80, 85]).reshape(-1, 1)
y = np.array([2.5, 3, 3.5, 3.8, 4, 4.5, 5])

# 2. Fit Linear Regression model
model = LinearRegression()
model.fit(X, y)

# 3. Prediction for given sizes
X_pred = np.array([55, 68, 76, 90]).reshape(-1, 1)
y_pred = model.predict(X_pred)

# Print results
for size, price in zip(X_pred.flatten(), y_pred):
    print(f"House size {size} m² → Predicted price: {price:.2f} tỷ")

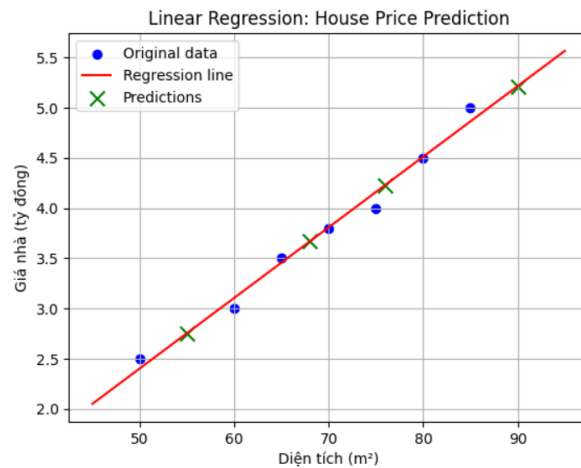
# 4. Visualization
# Scatter original data
plt.scatter(X, y, color='blue', label="Original data")

# Regression Line
x_line = np.linspace(45, 95, 100).reshape(-1, 1)
y_line = model.predict(x_line)
plt.plot(x_line, y_line, color='red', label="Regression line")

# Prediction points
plt.scatter(X_pred, y_pred, color='green', marker='x', s=100, label="Predictions")

# Labels and title
plt.xlabel("Diện tích (m²)")
plt.ylabel("Giá nhà (tỷ đồng)")
plt.title("Linear Regression: House Price Prediction")
plt.legend()
plt.grid(True)
plt.show()
|
```

House size 55 m² → Predicted price: 2.75 tỷ
 House size 68 m² → Predicted price: 3.67 tỷ
 House size 76 m² → Predicted price: 4.23 tỷ
 House size 90 m² → Predicted price: 5.21 tỷ



1.13.

```
[4]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. Dataset (Height in meters, Weight in kg)
# Đây là bộ dữ liệu hợp lý hơn (tham khảo từ sách ML):
heights = np.array([[1.60], [1.65], [1.70], [1.73], [1.80]])
weights = np.array([60, 65, 72.3, 75, 80])

print("\nOriginal Data (Height in meters, Weight in kg):")
for i in range(len(heights)):
    print(f"Height: {heights[i, 0]:.2f}m, Weight: {weights[i, 0]:.1f} kg")

# 2. Train Linear Regression model
model = LinearRegression()
model.fit(heights, weights)

# 3. Get coefficient and intercept (as scalars, not arrays)
gradient = model.coef_.item()
intercept = model.intercept_.item()
print(f"\nLearned Regression Line: Weight = {gradient:.2f} * Height + {intercept:.2f}")

# 4. Define new heights to predict
new_heights = np.array([[1.62], [1.78], [1.85]])
predicted_weights = model.predict(new_heights)

print("\nPredicted Weights for Specific New Heights:")
for i in range(len(new_heights)):
    print(f"For a height of {new_heights[i, 0]:.2f}m, predicted weight = {predicted_weights[i, 0]:.2f} kg")
```

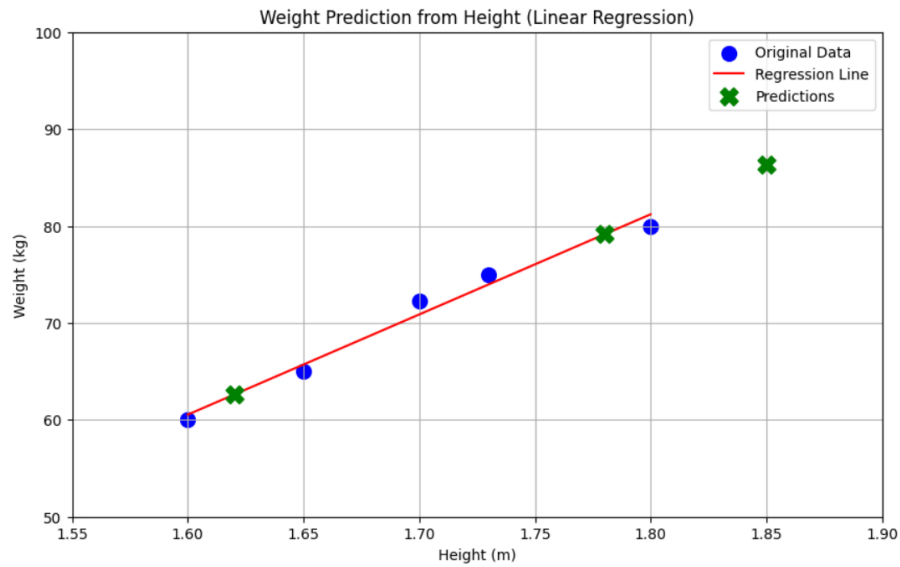
```
# 5. Visualization
plt.figure(figsize=(10, 6))
plt.scatter(heights, weights, color='blue', label='Original Data', s=100)
plt.plot(heights, model.predict(heights), color='red', label='Regression Line')
plt.scatter(new_heights, predicted_weights, color='green', marker='X', s=150, label='Predictions', zorder=5)

plt.title('Weight Prediction from Height (Linear Regression)')
plt.xlabel('Height (m)')
plt.ylabel('Weight (kg)')
plt.grid(True)
plt.legend()
plt.axis([1.55, 1.90, 50, 100]) # adjust axis range for clarity
plt.show()
```

Original Data (Height in meters, Weight in kg):
Height: 1.60m, Weight: 60.0 kg
Height: 1.65m, Weight: 65.0 kg
Height: 1.70m, Weight: 72.3 kg
Height: 1.73m, Weight: 75.0 kg
Height: 1.80m, Weight: 80.0 kg

Learned Regression Line: Weight = 103.31 * Height + -104.75

Predicted Weights for Specific New Heights:
For a height of 1.62m, predicted weight = 62.61 kg
For a height of 1.78m, predicted weight = 79.14 kg
For a height of 1.85m, predicted weight = 86.37 kg



1.14.

```
[2]: # Nguyễn Hoàng Tùng

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression

# 1. Load California Housing dataset
housing = fetch_california_housing(as_frame=True)
df = housing.frame

print("\nFirst 5 rows of the California Housing dataset:")
print(df.head())

# 2. Select features (e.g., MedInc = median income, AveRooms = average rooms per household)
X = pd.DataFrame(np.c_[df["MedInc"], df["AveRooms"]], columns=["MedInc", "AveRooms"])
y = df["MedHouseVal"]

print(f"\nSelected features (X) shape: {X.shape}")
print(f"Target (y) shape: {y.shape}")

# 3. Train Linear Regression model
model = LinearRegression()
model.fit(X, y)

# 4. Print regression equation
intercept = model.intercept_
coef_MedInc, coef_AveRooms = model.coef_
```

```

print("\nLearned Regression Equation:")
print(f"MedHouseVal = {intercept:.2f} + ({coef_MedInc:.2f} * MedInc) + ({coef_AveRooms:.2f} * AveRooms)")

# 5. Visualization (3D regression plane)
fig = plt.figure(figsize=(12, 10))
ax = fig.add_subplot(111, projection="3d")

# Scatter plot of actual data (sampled for clarity)
sample_idx = np.random.choice(len(X), 500, replace=False)
ax.scatter(X["MedInc"].iloc[sample_idx], X["AveRooms"].iloc[sample_idx], y.iloc[sample_idx],
           color="blue", label="Actual Data", s=20, alpha=0.6)

# Create meshgrid for regression plane
x_surf = np.linspace(X["MedInc"].min(), X["MedInc"].max(), 50)
y_surf = np.linspace(X["AveRooms"].min(), X["AveRooms"].max(), 50)
x_surf, y_surf = np.meshgrid(x_surf, y_surf)
z_surf = model.intercept_ + model.coef_[0]*x_surf + model.coef_[1]*y_surf

# Plot regression plane
ax.plot_surface(x_surf, y_surf, z_surf, color="red", alpha=0.4)

# Labels
ax.set_xlabel("MedInc (Median Income)")
ax.set_ylabel("AveRooms (Average Rooms)")
ax.set_zlabel("MedHouseVal (Median House Value in $100,000s)")
ax.set_title("Multiple Linear Regression: California Housing (MedInc + AveRooms)")

plt.show()

```

```

# 6. Example predictions
print("\nExample Predictions:")
new_data = pd.DataFrame({
    "MedInc": [3, 6, 9],
    "AveRooms": [4, 6, 8]
})
predictions = model.predict(new_data)

for i in range(len(new_data)):
    print(f"For MedInc={new_data['MedInc'][i]}, AveRooms={new_data['AveRooms'][i]} "
          f"> Predicted MedHouseVal = ${predictions[i]*100000:.2f}")

```

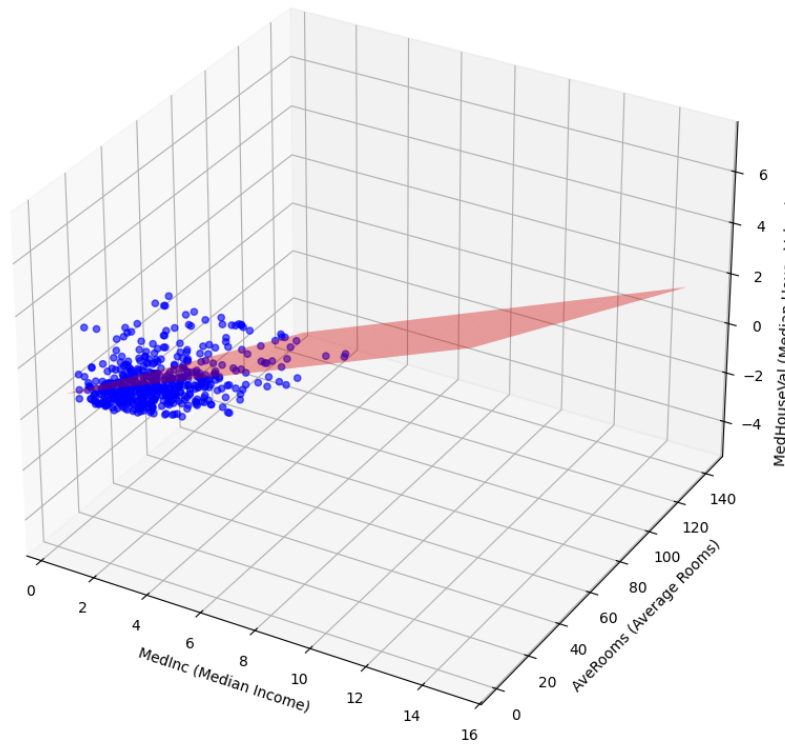
First 5 rows of the California Housing dataset:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23	4.526
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22	3.585
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24	3.521
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25	3.413
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25	3.422

Selected features (X) shape: (20640, 2)
Target (y) shape: (20640,)

Learned Regression Equation:
MedHouseVal = 0.59 + (0.43 * MedInc) + (-0.04 * AveRooms)

Multiple Linear Regression: California Housing (MedInc + AveRooms)



Example Predictions:

For MedInc=3, AveRooms=4 -> Predicted MedHouseVal = \$174500.51
For MedInc=6, AveRooms=6 -> Predicted MedHouseVal = \$297127.29
For MedInc=9, AveRooms=8 -> Predicted MedHouseVal = \$419754.07